

计算机系统基础实 验报告

实验一：Datalab



学生姓名：俞浩坤

学 号：22300240034

日 期：2023.10.7

一、实验结果：

在终端中执行./dlc -e bit.c 后的截图：

```
yuhaoshen@LAPTOP-QU516I60:/mnt/c/datalab-handout$ ./dlc -e bits.c
dlc:bits.c:148:tconst: 2 operators
dlc:bits.c:160:bitNand: 3 operators
dlc:bits.c:173:ezOverflow: 4 operators
dlc:bits.c:186:fastModulo: 4 operators
dlc:bits.c:198:findDifference: 8 operators
dlc:bits.c:214:absVal: 3 operators
dlc:bits.c:232:secondLowBit: 7 operators
dlc:bits.c:259:byteSwap: 17 operators
dlc:bits.c:280:byteCheck: 17 operators
dlc:bits.c:296:fractions: 5 operators
dlc:bits.c:318:biggerOrEqual: 15 operators
dlc:bits.c:342:hdOverflow: 18 operators
dlc:bits.c:370:overflowCalc: 19 operators
dlc:bits.c:391:logicalShift: 9 operators
dlc:bits.c:419:partialFill: 20 operators
dlc:bits.c:445:float_abs: 6 operators
dlc:bits.c:505:float_cmp: 26 operators
dlc:bits.c:568:float_pow2: 31 operators
dlc:bits.c:621:float_i2f: 23 operators
dlc:bits.c:674:oddParity: 32 operators
dlc:bits.c:714:bitReverse: 40 operators
dlc:bits.c:769:mod7: 51 operators
```

在终端中执行./btest 后的截图：

```
yuhaoshen@LAPTOP-QU516I60:/mnt/c/datalab-handout$ ./btest
Score   Rating  Errors  Function
1       1       0      tconst
2       2       0      bitNand
2       2       0      ezOverflow
3       3       0      fastModulo
3       3       0      findDifference
4       4       0      absVal
4       4       0      secondLowBit
5       5       0      byteSwap
5       5       0      byteCheck
5       5       0      fractions
6       6       0      biggerOrEqual
6       6       0      hdOverflow
7       7       0      overflowCalc
8       8       0      logicalShift
8       8       0      partialFill
3       3       0      float_abs
5       5       0      float_cmp
6       6       0      float_pow2
7       7       0      float_i2f
2       2       0      oddParity
2       2       0      bitReverse
2       2       0      mod7
Total points: 96/96
```

二、函数实现思路：

1、tconst(void): 注意到 0 取反后的结果为 0xFFFFFFFF，将其与 0x1F 即 31 异或即可得到 0xFFFFF0。

2、bitNand(int x, int y): 由于进行 $\sim(x \& y)$ 运算后，只有在 x 和 y 对应的二进制位都为 1 时结果中的位才为 0，其余情况均为 1，因而先将 x 和 y 取反后再进行或运算即可得到结果。

3、ezOverflow(int x, int y): 因为 x 和 y 均为有符号二进制正数，因而只有当相加后结果的最高位为 1，即结果为负数时才会发生溢出，只需返回最高位即可，将结果右移动 31 位再进行取反和取非运算即可得到最高位。

4、fastModulo(int x, int y): 先将 1 左移 y 位得到 2 的 y 次方，由于二进制中每一位的权重数值上都是其右边一位的两倍，因而比 2 的 y 次方高的位数的权重都能被其整除，只需保留 x 中的 0 到 y-1 位即可，将 2 的 y 次方减去 1 再与 x 相与就能得到答案。

5、findDifference(int x, int y): 题目要求返回的是 x 和 y 异或后的结果，将 $x|y$ 的结果取反后将都为 0 的位数字 1，再将 $\sim x|\sim y$ 的结果取反后将都为 1 的位数字 1，其余位均置 0，将两者结果相或再取反即可得到异或后的结果。

6、absVal(int x): 将 x 的最高位右移 31 位，如果 x 为负数得到全 1 (数值上为 -1)，为正数得到全 0，将 x 与之相加后如果 x 为正数则不变，为负数则减去 1，再进行异或运算，由于与全 0 做异或得到其本身，与全 1 做异或相当于取反，由于负数在之前已经减去了 1 因而取反后不需要再加 1 就能得到其相反数。

7、secondLowBit(int x): 注意到将 x-1 后可以将 x 最低位的 1 变为 0 而更高位的 1 不变，因而将 x 与 x-1 相与将最低位的 1 消去，再把结果减去 1 后取反保留第二低的 1 并把更高位的 1 消去，最后把结果与之前的结果相与把由于取反多出来的 1 消去即可得到答案。

8、byteSwap(int x, int n, int m): 先将 m 和 n 分别左移 3 位得到所需交换的两个字节最低位的编号，之后将 x 按这些编号右移并将结果与 0xFF 相与即可得到所需交换的两个字节，将这两个字节按之前存储的最低位编号右移可得到它们在 x 中的表示，再按对方的最低位编号右移可得到它们在结果中的表示，让 x 减去之前两个字节的表示再加上结果中两个字节的表示即可得到答案。

9、byteCheck(int x): 对第 0 个字节左移 24 位，其余字节先左移到底再右移到底，如果字节全为 0 则得到的结果应该为 0，否则不为 0，对每个结果取两次非即可使全为 0 的部分变为 0，不全为 0 的部分变为 1，再将其相加即可得到答案。

10、fractions(int x): 先将两个 x 之和的结果存进 ttemp，再将两个 ttemp 之和存进 tttemp 即得到 4 个 x 之和，将三个数相加即得到 7 个 x 再右移四位就是结果。

11、biggerOrEqual(int x, int y): 首先将 x 和 y 右移 31 位之后都取非，如果为正数得到 1，为负数得到 0，如果 x 为正 y 为负则结果为 1，反之为 0，将结果分别存入 result 和 rresult，之后计算 x-y 的结果并用同样的方法判断正负，因为可能发生溢出需要与一开始的判断结果进行合并才能得到答案。

12、hdOverflow(int x, int y): 分别将 x, y, x+y 右移 31 位后两次取非可以得到 x, y, x+y 的最高位，如果 x 和 y 最高位不同或者 x, y, x+y 三者最高位都相同的话则没发生溢出，否则发生溢出，由此写出语句进行判断。

13、overflowCalc(int x, int y, int z): 先生成一个 0xFFFF 的常量能通过与其相与得到二进制数的 0-15 位，将 x, y, z 分别右移 16 位然后取其结果的 0-15 位接着相加

得到前 16 位数的相加结果，接着直接取 x, y, z 的 0-15 位进行相加，由于 0-15 位相加结果对前 16 位相加结果溢出产生影响的只有结果的 16-17 位，将 16-17 位取出再与前 16 位相加结果相加，最后将结果的 16-17 位取出即为答案。

14、`logicalShift(int x, int n)`: 首先将 x 右移 31 位并与 1 相与得到 x 的最高位，再将最高位左移 31 位与 x 相加使 x 最高位变 0，之后再对 x 进行右移能使 x 右移时补 0，最后再把 x 的最高位按右移之后的位数左移加到结果里即为答案。

15、`partialFill(int l, int h)`: 由于要确保左移和右移的位数不超过 31，所以对于 h 只能取 $31-h$ 的结果，首先让 $l+(l\&1)$ 变为比 l 大的最小偶数，由于 $31-h$ 使 h 奇偶性改变，因而采用 $h|1$ 将 h 变为比 h 大的最小奇数，为了更快的填充偶数位的 1 采用 `0x55` 作为辅助，通过将其左移 `evenl` 和 `evenl+8` 位使比 l 大 16 位之内的偶数位置 1，再将 `0x55` 左移 24 位后右移 $31-h$ 和 $31-h+8$ 使比 h 小 16 位之内的偶数位置 1，由于 l 只能决定 0-15 位的偶数位而 h 决定 16-31 位的偶数位，因而分别将两者的结果和 `0xFFFF` 和 `0xFFFF0000` 相与后相加得到答案。

16、`float_abs(unsigned uf)`: 通过将 uf 与 `0x7FFFFFFF` 相与得到将 uf 首位变为 0 的结果，如果 uf 是数字即为答案，接着通过将 uf 与 `0x7F800000` 和 `0x007FFFFFFF` 相与得到其指数位和尾数位，如果指数位为 255 且尾数位不为 0 则不是数字返回原数，否则返回之前的结果。

17、`float_cmp(unsigned uf1, unsigned uf2)`: 用和上题一样的方法将 $uf1$ 和 $uf2$ 的符号位、指数位和尾数位取出，首先判断 $uf1$ 和 $uf2$ 中有没有不是数字的，如果有就直接返回 0，接着判断符号位，如果 $uf1$ 为负 $uf2$ 为正则一定不大于 $uf2$ 返回 0，如果 $uf1$ 为正且 $uf2$ 为负且不为 0 则一定大于 $uf2$ ，判断完后只剩下两者正负相同的情况，分别比较指数位和尾数位的大小即可得到答案。

18、`float_pow2(unsigned uf, int n)`: 首先将 uf 的符号位、指数位和尾数位全部取出，然后分情况讨论，如果指数位为 255 则不需要考虑 n 直接返回 uf ，如果指数位不为 255 且不为 0，则将指数位与 n 相加，如果相加结果大于等于 255 则返回符号位对应的正负无穷，不然则将指数位加上 n 后返回原数，如指数位为 0 则在乘法时先左移尾数位，等到尾数位最大的 1 移动到指数位上时停止，判断移动了多少次，如果移动次数大于 n 则只移动尾数位，小于 n 则再分情况讨论，如果 n 为 255 且只移动一次则得到符号位对应的正负无穷，否则得到尾数位移动后加上指数位减去尾数位移动次数再加上 n 最后加上符号位的结果。

19、`float_i2f(int x)`: 首先将 x 的符号位取出，接着如果 x 位 0 则直接返回 x ，如果 x 小于 0 则将 x 变为其相反数便于计算，接着将 x 不断左移直到 x 的 31 位为 1，通过移动次数来得到结果的指数位，接着取出 x 左移后的最后 9 位来判断是否需要进位，如果最后 9 位最高位为 0 或者最高位为 1 但后 8 位全为 0，且第 10 位为 0 则不进位，其余情况均进位，将 x 再右移九位后加上符号位、指数位和进位结果即得到答案。

20、`oddParity(int x)`: 注意到 x 中奇数位和偶数位的和即是这两位中 1 的数量，因而将 x 相邻一位相加后，相邻两位相加，相邻四位相加，相邻八位相加，相邻十六位相加后可得到 1 的数量，对于相邻一位相加构造 `0x55555555`，将 x 与其相与并加上 x 右移 1 位后与其相与的结果中每两位存储了相邻两位中 1 的个数，对于相邻两位相加构造 `0x33333333` 采取相同的做法，考虑到由于每四位中最多只有四个 1 因而四位第一位必为 0，因而在相邻四位相加时可以先计算右移四位后的相加结果再与 `0x0F0F0F0F` 相与来节省操作符，同样在相邻八位时也能如此，在相邻八位时构造的数为 `0x00FF00FF`，而在相邻十六位相加时因为最后需要计算

的结果是 1 的个数的奇偶性，因而只需相加后取出第 0 位取非即可，不需要再进行相与，由此可得到结果。

21、bitReverse(int x): 同上题一样，先将 x 的相邻一位取反，之后相邻两位取反，相邻四位取反，相邻八位取反，相邻十六位取反即可得到答案，相邻一位取反仍然构造 0x55555555，将 x 与之相与后左移 1 位再加上 x 右移一位后与之相与的结果即可使相邻一位取反，相邻两位取反时构造 0x33333333 采用相同做法，由于这次 32 位数中每一位都要保留，所以不能像上一题一样节省字符，在相邻四位取反时仍需要构造 0x0F0F0F0F 再重复相同操作，相邻八位时构造 0x00FF00FF 重复同样操作，相邻十六位时需要构造 0x0000FFFF 重复相同操作，由于在左移时低位自动补 0 因而可以不进行相与来节省一个操作符，之后可得到结果。

22、mod7(int x): 注意到从 x 的第 0 位开始每三位 mod7 的结果都是 1, 2, 4 循环，因而每次通过对 x 取 0-2 位然后右移 3 位再将结果相加所得的数 mod7 与原来结果相同，注意 x 的最高位权重为-2，先将其变为 5，并将 x 右移 31 位提取 x 的最高位并于~6 相与，如果最高位为 0 则得到 0，最高位为 1 则得到-7，接着因为第一轮得到的结果最大为 75，因而在重复一轮操作时只需右移两次，这次得到的结果最大为 14，再进行一轮这次只需右移一次，得到结果最大为 7，接着判断结果是否为 7，将结果减去 7 后右移 31 位，如果结果相与 7 则得到全 1，相与后为原数，否则为全 0 相与后得到 0，接着将结果与之前 x 的最高位取得的结果相加，如果 x 为负数则使结果-7 否则不变，最后为了防止出现-7 再对之前减去 7 的结果做一次相同操作使-7 变为 0 其余不变，即得到答案。